

SPETC v10 — User Guide

Mauro Barbieri

mauro.barbieri@pm.me

2007–2026

Abstract

This guide explains how to install, configure and operate `SPETC`, the spectro-photometric exposure time calculator, from the first launch to photometric and spectroscopic planning, results interpretation and CSV export. No knowledge of the internal physics is required; the companion document *Physics of the ETC* covers the models and formulae.

Contents

1	Getting started	2
1.1	What <code>SPETC</code> is and what it computes	2
1.2	Requirements	2
1.3	The data directory	3
1.4	First launch and configuration persistence	3
1.5	A first calculation, explained	3
2	The main window	4
2.1	Observation, site and target	4
2.2	Telescope, detector and conditions	4
2.3	Calculation settings	5
2.4	Filter and template selection	5
2.5	Run actions	5
3	Photometry step by step	6
3.1	Point sources	6
3.2	Extended sources: galaxies, nebulae, planets	6
3.3	Target S/N mode and the stack planner	7
3.4	Differential photometry (variable stars, exoplanets)	7
4	Spectroscopy step by step	7
4.1	Slit spectroscopy	7
4.2	Slitless spectroscopy and the Star Analyser	8
4.3	Getting a science answer: S/N, $\sigma(\text{EW})$ and the stack plan	9
5	Results, plots and export	9
5.1	The Results window	9
5.2	Scientific plots	9
5.3	Practical notes	9

6	Input file formats, in full	10
6.1	The template catalogue: <code>data/interpola.db.csv</code>	10
6.2	Template spectra	11
6.3	The filter list and profiles	11
6.4	Detector and instrument curves	12
6.5	The atmospheric transmission FITS file	13
6.6	Sites: <code>etc_sites.json</code> and <code>observatories.json</code>	13
6.7	Instrument profiles	13
6.8	Horizon profile CSV	13
6.9	Batch run descriptions	14
7	Command-line tools	14
7.1	Calibrating your own filters: <code>make_filter_profile.py</code>	14
7.2	Importing template spectra: <code>add_template.py</code>	14
7.3	Headless runs: <code>spetc_batch.py</code>	14
8	Troubleshooting	15

1 Getting started

1.1 What SPETC is and what it computes

SPETC answers the two questions every observation begins with: *what signal-to-noise ratio will I reach in a given exposure*, and *how long must I expose to reach the signal-to-noise ratio I need*. It answers them for two kinds of measurement: broad-band aperture photometry (measuring the brightness of an object through a filter) and spectroscopy (dispersing the light with a slit spectrograph or a slitless grating such as the Star Analyser and measuring the spectrum). The program is not tied to any particular observatory: you describe *your* telescope, *your* camera and *your* sky, and the program predicts what that combination will deliver on a target of a given brightness and spectral type, at a given place on the sky, at a given time of a given night.

The prediction is physical, not statistical: the program starts from an absolutely calibrated template spectrum of a star similar to your target, scales it to the magnitude you enter, pushes that spectrum through every element of the light path — the atmosphere at the current airmass, the telescope aperture, the filter, the optics, the detector quantum efficiency — and counts the photo-electrons that arrive in your aperture or in each spectral resolution element. It then adds every noise source that a real detector and a real atmosphere contribute (photon noise from the object and the sky, dark current, read noise, atmospheric scintillation, analogue-to-digital quantization) and reports the resulting signal-to-noise ratio, along with a saturation prediction for the brightest pixel. The companion document *Physics of the ETC* derives every formula; this guide tells you which buttons to press and what the numbers mean.

1.2 Requirements

You need Python 3.10 or newer with Tk support. Tk (the graphical toolkit) is included in the standard installers from python.org on Windows and macOS; on Linux it is usually a separate package (for example `sudo apt install python3-tk` on Debian/Ubuntu or `sudo dnf install python3-tkinter` on Fedora). Everything else is installed with pip. In a terminal, change into the directory where you unpacked SPETC and run:

```
python3 -m pip install -r requirements.txt
python3 etc_gui.py
```

The first command installs the scientific libraries the program is built on (NumPy, SciPy, pandas, Matplotlib, Astropy and astroquery). The second starts the program. On Windows, replace `python3` with `py`.

1.3 The data directory

Beside `etc_gui.py` there must be a directory called `data`. It is the program's library and ships ready to use; you never need to edit it to get started, but you will add to it as you customise the program, so it is worth knowing what lives inside:

`interpol.db.csv` The *template catalogue*: one line per stellar template, giving its name, spectral type, brightness calibration and the file that holds its spectrum. The template list you see in the GUI is this file, read at start-up. Its exact format is specified in Sect. 6.1.

`bpgs/` The template spectra themselves: absolutely calibrated stellar spectra from the Bohlin CALSPEC/BPGS libraries, one file per star, covering the main spectral types from O to M plus the Sun and Vega. You can add your own (Sect. 6.2).

`filters.list` and `filters/` The list of available filters and their transmission profiles in the Spanish Virtual Observatory (SVO) XML format — Johnson–Cousins, Sloan, 2MASS and others, each carrying its own photometric calibration. The special `BLANK.xml` profile represents “no filter at all” (Sect. 6.3).

`qe.dat` The default detector quantum-efficiency curve: how many electrons the sensor produces per incoming photon, as a function of wavelength (Sect. 6.4).

`earth_atmospheric_transmission.fits` Optional: a measured atmospheric transmission table (the shipped one is the Paranal extinction product). When absent, a built-in broad-band extinction table is used instead.

Two more files live beside the program itself rather than in `data`: `etc_sites.json`, your personal list of observing sites, and `etc_user_config.json`, the saved state of every control in the window. Both are created and updated automatically.

1.4 First launch and configuration persistence

On the first start you get the default configuration: the Asiago (Cima Ekar) observatory, a 358-mm telescope, and Vega as the target template. Every control in the window — every number, every selector — is saved to `etc_user_config.json` after each successful calculation and when you quit, and restored on the next start. There is no “save settings” button because saving is automatic; to transfer your setup to another computer, copy that file (and your instrument profile, see Sect. 6.7) along with the program.

1.5 A first calculation, explained

The fastest way to understand the program is to run it once. The following walk-through predicts the photometry of a 10th-magnitude star tonight.

First, tell the program *where* you are. In the leftmost column, open the site selector and choose one of the built-in observatories, or type your latitude (degrees, north positive), longitude (degrees, east positive) and elevation (metres) directly into the three fields. The site matters more than it may seem: the latitude and longitude determine when and how high your target rises, the elevation sets the atmospheric pressure used for refraction and the air mass of turbulence responsible for scintillation noise.

Second, tell it *when* you observe. Set the date and the UT time. If you think in local time, note the timezone selector: choose an IANA timezone name such as `Europe/Rome` and the program handles daylight saving for you in all displayed local times; the calculation itself always runs on UT.

Third, tell it *what* you observe. Type the target coordinates into the RA and Dec fields. Both decimal degrees (279.2347) and sexagesimal (18:36:56.3) forms are accepted. If you know only the object's name, type it into the name field and press **Resolve name**: the program queries the SIMBAD database (this is the only feature that uses the internet) and fills the coordinate fields for you. Then enter the target's magnitude — say 10.0 — and check that the *reference filter* selector says in which band that magnitude is expressed (V, by default) and in which system (Vega, by default).

Fourth, choose a template. In the template list, pick a star of roughly the same spectral type as your target — for a Sun-like target choose a G2V, for a hot star a B or A star. The template provides the *shape* of the spectrum; your entered magnitude provides the level. The panel shows a preview of the selected spectrum so you can see what you picked.

Finally, press the green Run ETC button. Three things appear: the **Results window** with the numbers (Sect. 5), the **plot window** with S/N versus time across the night, the altitude track and the sky path of the target, Sun and Moon, and the status line reporting completion. The red marker in the plots is the time you selected; the curve around it shows how the same observation would fare earlier or later in the night, with the sky brightness, the airmass and the Moon recomputed at every time step.

If the button is grey rather than green, the program is telling you that the current combination cannot be computed — most often because the selected template does not overlap the selected filter. The message above the response preview says exactly what is wrong; fix the selection and the button turns green.

2 The main window

The window is organised in five columns, left to right.

2.1 Observation, site and target

Site selector (built-in observatories merged with your persistent `etc_sites.json`; **Save current site** and **Import site** file manage it), geographic coordinates and elevation, date and UT time of the planned observation, the observing-interval selector, the time zone (IANA name with DST, e.g. Europe/Rome, or a fixed UTC offset), and the target: RA/Dec fields (decimal degrees or hh:mm:ss / dd:mm:ss) with the optional SIMBAD **Resolve name** button. Running the ETC itself never uses the network: only the coordinates in the fields count.

The OBSERVATORY panel also hosts the **horizon-profile generator**. Enter a search radius (1–100, default 10, in km or miles via the unit selector) and press **Generate horizon profile**: the program downloads the Copernicus GLO-30 elevation model around your coordinates (cached locally, so only the first run of an area needs the internet), and traces the apparent terrain horizon — mountains, hills, the curvature of the Earth and standard refraction included — in 360 azimuth steps. The computation runs in the background and reports the highest obstruction when done; the profile is *automatically* saved as a CSV in `data/horizons/` (format in Sect. 6.8). **Display horizon plots** the saved profile for the current coordinates at any time. The generator needs two extra libraries (`pip install rasterio pyproj`); everything else in the program works without them.

2.2 Telescope, detector and conditions

Telescope diameter, central obstruction, focal length and scalar optics throughput; optional calibrated instrument-response and slit resolving-power curves (two-column files with an explicit wavelength unit); detector pixel size, gain, full well, ADC bit depth, read noise and dark current; the **sensor type** (mono, or osc with the R/G/B channel: aperture rates and pixel counts scale by the Bayer fill fraction, saturation stays per physical pixel; use the channel's effective QE curve); a **CMOS gain table** (a text file with one row per gain setting: setting, e^- /ADU, read noise, full well — published by camera makers and measured on PhotonsToPhotos): pick the setting and the three detector fields fill consistently; seeing FWHM; the sky-background mode (`ing` = physical night/twilight/day model, `fixed_ab` = your own observed sky surface brightness, `sqm` = your **zenith SQM reading** in V mag/arcsec², with a **Bortle class** preset that fills

a typical value; band colours and the Moon model are applied on top); and the **PSF model** (`gaussian` or `moffat` with its β ; Moffat reproduces the wings of real star images and is recommended); an optional **seeing-scaling** checkbox that makes the seeing vary with wavelength and airmass (Kolmogorov turbulence: blue light more blurred than red, and worse as the target sets — with it ticked the value you enter is the zenith V seeing); an optional **extra background** in $e^-/s/pixel$, a catch-all for detector glow, stray light or ghosting that raises the background noise and the peak-pixel prediction; a **guiding rms** in arcsec (0 = perfect tracking) that adds image-motion blur in quadrature to the seeing — enter your autoguider’s rms error, or an estimate of the unguided drift per exposure; a **telluric bands** checkbox that adds the O_2/H_2O molecular absorption bands of the red and near-infrared (the deep A band at 7605 Å in particular) on top of the smooth extinction curve; and the **sky annulus pixels** field: the number of pixels you actually use to measure the background around the star (0 = assume it perfectly known). A finite value adds the classical $(1 + n_{pix}/n_{sky})$ sky-subtraction noise to every background term — with a small annulus this is tens of percent of the background noise, so faint-target predictions become noticeably more honest. **Save/Load** instrument profile stores the whole column as a portable JSON with its QE and response curves.

2.3 Calculation settings

Photometry or spectroscopy; exposure time; optional **target S/N** (when set, the ETC returns the required exposure instead, flagging saturation — the solver includes scintillation, so bright-star requests are honest); the **stack sub-exposure** preference (0 = automatic) feeding the stack planner, whose result appears in the label below after each run. The photometry panel adds the aperture radius, the **source geometry**: `point`, or `extended` with the source area in $arcsec^2$ for galaxies, nebulae and planets (the magnitude you enter stays the *total* integrated magnitude), and the optional **comparison-star magnitude** for differential photometry — the mmag-per-frame precision appears in the label below. The spectroscopy panel holds the resolving power, slit width, extraction height, sampling, the slitless parameters (Sect. 4), the wavelength range and the S/N reference wavelength, the **slit orientation** (`parallactic` or `fixed`), and the **spectrograph geometry helper**: enter grating lines/mm and collimator/camera focal lengths and press **Compute R from geometry** — the Littrow grating equation fills the R field (which stays editable) and reports whether you are slit- or seeing-limited. The optional **clamp** checkbox below it goes one step further: with it ticked, the engine itself limits the entered R to what the geometry can deliver, so a typo like $R = 100\,000$ on a 600 l/mm grating cannot silently inflate every prediction.

2.4 Filter and template selection

Two independent filters: the **reference** filter, in which your magnitude is expressed (Vega or AB), and the **observing** filter, physically in the beam for both photometry and spectroscopy (spectroscopy defaults to `BLANK`, the unfiltered response). The panel previews the observing transmission curve with its pivot wavelength, width and zero point; the template selector searches the catalogue by name and previews the selected flux distribution; **Validate V and B–V** checks the template’s synthetic colours against the catalogue. Two optional fields transform the template before it is calibrated: the **radial velocity** in km/s (positive receding) shifts all wavelengths by $(1 + v/c)$ — relevant for high-velocity stars and extragalactic targets at low resolution — and **E(B–V)** reddens the template with the standard CCM89 interstellar extinction law ($R_V = 3.1$). The magnitude you enter always stays the *observed* one; the transforms only change the template’s colours, exactly as reddening and motion do to a real star.

2.5 Run actions

Run ETC, the live observatory status (Sun/Moon altitude, current airmass of the target), and Exit (which saves the configuration).

3 Photometry step by step

3.1 Point sources

Step 1 — declare the geometry. Set `Source geometry` to `point`. In this mode the program treats your target as an unresolved star whose light on the detector is spread by the atmosphere into the seeing profile you selected (Gaussian or Moffat). Everything that follows — how much light your aperture captures, how bright the central pixel gets — is computed from that profile.

Step 2 — choose the aperture radius. This is the radius, in arcseconds, of the circular software aperture you will later use in your photometry program (AstroImageJ, ASTAP, IRIS, ...); the ETC mirrors it. The choice is a genuine trade-off. A large aperture captures nearly all the star’s light but also more sky background and more pixels’ worth of read noise; a small one keeps the noise down but loses starlight in the seeing wings — and with a Moffat profile the wings carry more light than intuition suggests. As a rule of thumb, a radius of $1.5\times$ the seeing FWHM captures about 95% of a Moffat profile; but you do not need to optimise by hand, because the ETC applies the exact encircled-energy correction for whatever radius you type, so every choice gives a correct (if not optimal) prediction. If you want the optimum, try a few values and watch the S/N.

Step 3 — enter the brightness. Type the target magnitude, and verify the two selectors that give that number its meaning: the *reference filter* (the band your magnitude is expressed in) and the *magnitude system* (Vega or AB). These need not match the filter you observe through: it is perfectly normal to say “the target is $V = 12.3$ ” while observing in R — the program derives the colour correction from the template’s spectrum. What matters is that the magnitude, band and system agree with wherever you got the number from (catalogues usually state Vega magnitudes for Johnson bands and AB for Sloan bands).

Step 4 — run and read. Press Run ETC. The headline number is the S/N at your selected time, but look at the whole S/N-versus-time curve in the plot window: it shows the observation degrading as the target sets and the airmass grows, and brightening sky as the Moon rises. The Results window details every contribution (Sect. 5).

One behaviour surprises most users the first time: make the star very bright and the S/N stops improving, saturating at a few thousand regardless of magnitude. This is not a bug — it is atmospheric scintillation, the twinkling of stars, which contributes a noise proportional to the signal itself and therefore sets a hard ceiling per exposure. No aperture photometry from the ground beats it in a single frame; the way past it is stacking many frames, which is what the stack planner quantifies.

3.2 Extended sources: galaxies, nebulae, planets

Set `Source geometry` to `extended` and fill in the source area in arcsec^2 . The convention to remember is that *the magnitude you enter remains the total, integrated magnitude of the object* — the number catalogues quote — and the program spreads it uniformly over the area you declare. Some example areas: a galaxy listed as $2' \times 1'$ covers $\pi ab/4 \simeq 5\,650 \text{ arcsec}^2$ if those are full axes; Jupiter at opposition ($\simeq 44''$ diameter) covers $\simeq 1\,500 \text{ arcsec}^2$; the Ring Nebula ($\simeq 86'' \times 62''$) about $4\,200$.

Physically the program then does three things differently from a point source. First, your aperture no longer captures “all the light minus seeing losses” but simply the fraction of the object’s area it covers — an aperture smaller than the galaxy sees proportionally less of it. Second, there is no seeing concentration: the brightest pixel holds the mean surface brightness, not a stellar peak, which typically moves the saturation limit by orders of magnitude (this is why you can expose far longer on a galaxy than on a star of the same total magnitude). Third, the S/N refers to the light inside your aperture, which is the quantity your photometry software will actually measure. The approximation declared in the model — uniform brightness — is good when the object is much larger than the seeing disc; for objects comparable to the seeing (small planetary nebulae), treat the result as indicative.

3.3 Target S/N mode and the stack planner

Typing a number into Target S/N reverses the question: instead of “what S/N in this exposure” the program answers “what exposure for this S/N”. The inversion is done in closed form with the complete noise budget — including scintillation, which matters enormously here: for a bright star a solver without scintillation would confidently prescribe a short exposure that can never reach your target, no matter what the photon statistics say. If the S/N you request lies above the scintillation ceiling for a single frame, the required exposure grows until the ceiling (which rises as $\sqrt{t}$) meets it.

The required exposure, however, frequently collides with saturation: a bright target may need 300 s for your S/N while its peak pixel saturates in 20. This is exactly the situation the **stack planner** resolves. After every run with a target S/N, the label under CALCULATION reports a complete plan of the form:

Stack plan (aperture): 16×19 s (sub limited by saturation) = 304 s total for S/N 500; read-noise penalty +2.1% vs one ideal exposure. Background beats read noise above 45 s/frame.

Reading it: the longest safe single frame is 19 s (set by the brightest pixel; or by your own preference if you typed one into Stack sub-exposure); sixteen such frames reach the target S/N; splitting the exposure costs only 2.1% extra total time compared with one hypothetical long frame, because each frame adds its own read noise; and frames longer than 45 s would be sky-limited, meaning read noise no longer matters much beyond that length. If the penalty is large, your frames are too short for your read noise — raise the sub-exposure or use a lower-noise gain setting.

3.4 Differential photometry (variable stars, exoplanets)

Photometric monitoring is almost never absolute: you measure your target *against* a comparison star in the same field, and the number that matters is the scatter of that difference, in millimagnitudes. Enter the comparison star’s magnitude in Comparison star mag and the program runs the identical calculation for it, combines the two error budgets, and reports, in the PHOTOMETRY label and the outputs tab:

Differential precision vs $m=11.5$ comparison: 3.2 mmag per 60 s frame (target S/N 520, comparison S/N 610).

Both stars’ budgets include their own scintillation, treated as uncorrelated between the two — a conservative assumption at the arcminute separations of typical comparison stars. To judge feasibility: a typical exoplanet transit is 5–30 mmag deep, a bright-star transit like HD 189733 about 25 mmag; with 3.2 mmag per frame you resolve such a transit comfortably, and binning N frames improves the precision by $\sqrt{N}$. The AAVSO reporting convention likewise wants the per-frame uncertainty — this number is exactly that.

4 Spectroscopy step by step

4.1 Slit spectroscopy

Step 1 — establish the resolving power. Everything in a spectroscopic prediction hangs on the resolving power $R = \lambda/\Delta\lambda$, because it fixes how the light is divided among resolution elements. You have three ways to supply it, in increasing order of fidelity. If you know R from your spectrograph’s documentation, type it into Slit resolving power R and you are done. If you know your *hardware* instead — most amateurs do — fill the three geometry fields (grating grooves per mm, collimator focal length, camera focal length) and press Compute R from geometry: the program combines them with the telescope focal length, your slit width and the seeing through the grating equation, fills the R field, and tells you whether the resolution is limited by the slit or by the seeing — if it says “seeing-limited”, a narrower slit would only lose light without sharpening the spectrum. Finally, if you have a *measured* R -versus-slit-width curve

for your instrument, load it in the telescope column and it silently overrides the nominal value within its calibrated range.

Step 2 — describe the extraction. The slit width (arcsec) controls how much starlight enters; the extraction height (arcsec) is the length along the slit that your reduction software will sum over. Both cost and gain: a wide slit and tall extraction capture more of the seeing-blurred star but admit proportionally more sky, and the ETC accounts for both directions exactly, using the PSF model you selected. **Pixels per resolution element** is your detector sampling (typically 2–3); it fixes how many pixels’ worth of read noise each spectral bin carries.

Step 3 — decide the slit orientation. Atmospheric refraction disperses every star into a tiny vertical spectrum: at airmass 1.5 the blue image at 4000 Å sits about 1'' from the red image at 6600 Å. If your slit is aligned with that vertical direction — the *parallactic angle*, reported for every time sample in the Results window — all images fall along the slit and nothing is lost. If the slit sits at any other angle, the blue and red images drift out of it sideways. Choose *parallactic* if you rotate your slit during the night (or observe near the meridian, where the effect vanishes); choose *fixed* if your slit angle is fixed, and the program will charge the worst-case wavelength-dependent loss — tens of percent at the blue end at airmass 1.5–2, exactly the classical Filippenko effect. Comparing a run in each mode shows you precisely what slit rotation would buy.

Step 4 — set the wavelength range and reference. The range (min/max, Å) is the span the spectrum table will cover; the S/N reference wavelength is the single wavelength at which the time-series curve and the stack plan are evaluated — put it where your science is (H α at 6563 for emission-line work, 5500 for general use).

Step 5 — run and read. The plot window adds two spectral panels: the detected source rate and the S/N, both *per resolution element*, with a manual time slider that recomputes the whole spectrum at other times of the night. The Results table gives the full wavelength-by-wavelength budget (Sect. 5).

4.2 Slitless spectroscopy and the Star Analyser

In slitless spectroscopy there is no slit: every star in the field is dispersed directly, resolution is set by the seeing rather than by a slit, and no slit losses occur — which is why a Star Analyser on a modest telescope reaches surprisingly bright-looking spectra, at the price of low resolution and of sky light entering at every wavelength.

Mode *slitless* offers two ways to describe the disperser. For a **Star Analyser 100 or 200** (or any transmission grating in the converging beam), enter the groove density (100 or 200 lines/mm) and the distance from the grating to the sensor in millimetres — for a filter-wheel mounting this is roughly the wheel-to-sensor distance; measure it once. From these two numbers and your pixel size the program computes the dispersion itself: an SA100 at 42 mm on 4.63 μ m pixels gives 11.0 Å/pixel, matching the published calibration. The optional grating efficiency (a realistic first-order value is ~ 0.5) scales source and sky alike. Alternatively, leave the groove density at 0 and type a known dispersion in Å/pixel directly.

The resolution element is computed from the seeing, the intrinsic grating blur and the atmospheric-dispersion smear combined; with 2–3'' seeing expect $R \sim 100$ –200, and note that longer exposures do *not* sharpen it — only better seeing or a longer grating distance do. The cross-dispersion extraction height plays the role the extraction height plays in slit mode.

One consequence of having no slit deserves emphasis, because it changes the numbers a lot: **every pixel of a slitless spectrum receives sky light from the entire bandpass**, not just from the small wavelength interval mapped to it (the dispersed image of a uniform sky is still uniform). The ETC accounts for this in full since v10.2 — for a typical Star Analyser configuration the background is about a hundred times larger than a slit-style calculation would suggest. In practice: under a dark sky short exposures on bright targets work exactly as Star Analyser users know they do, but under a bright (moonlit or urban) sky the mode degrades much faster than a slit spectrograph, and the ETC now shows that honestly. Slitless results remain flagged *experimental* until validated against a calibrated instrument; treat absolute count levels with one raised eyebrow and relative comparisons with confidence.

4.3 Getting a science answer: S/N, $\sigma(\text{EW})$ and the stack plan

All spectroscopic counts and S/N are **per resolution element**, not per pixel — when comparing with other tools, check which convention they use, since the difference is a factor $\sqrt{N_{\text{pix}}}$. Each result row also carries $\sigma(\text{EW})$, the equivalent-width uncertainty at that wavelength from the Cayrel relation, in milli-Ångström. This converts S/N into the language of actual measurements: a line is securely measurable when its expected equivalent width exceeds roughly $3\sigma(\text{EW})$. Concretely, with $\sigma(\text{EW}) = 15 \text{ mÅ}$ at $\text{H}\alpha$, the 50–500 mÅ emission variations of a Be star are easy science, a 100 mÅ interstellar line is solid, and a 20 mÅ photospheric line is out of reach — one number, read directly from the table, settles what your night can and cannot do. The value at the reference wavelength is repeated in the label under PHOTOMETRY and in the outputs tab, and the stack planner works exactly as in photometry, applied to the reference resolution element.

5 Results, plots and export

5.1 The Results window

Three tabs:

- **Selected-time result:** the numerical table for the chosen time (magnitude row in photometry; per-wavelength rows in spectroscopy), showing *every* quantity the engine produced: rates, S/N, $\sigma(\text{EW})$ [mÅ] for spectroscopy, the scintillation and ADC-quantization noise in electrons, pixels in the aperture, peak electrons/ADU, saturation cause, maximum unsaturated exposure, the standard and instrumental magnitudes and the sky brightness used. **Save result CSV** exports the full table.
- **Time series:** one row per time sample with UTC and local time, MJD, elevation, azimuth, **parallactic angle**, airmass (Pickering, refraction-corrected), S/N or required exposure, **sky surface brightness** actually used, band-effective extinction and its per-airmass coefficient, the absorption-adjusted apparent magnitude, saturation flag, peak electrons and maximum unsaturated exposure. This table is the CSV export.
- **Assumptions / outputs:** a plain-language statement of every physical assumption of the current run — sky model and its components (including the SQM value in `sqm` mode), PSF model, the CMOS gain-table setting in use, the spectrograph geometry, noise terms, saturation policy — plus the selected-time altitude, airmass, parallactic angle and sky brightness, followed by a **SELECTED-TIME OUTPUTS** block: S/N, scintillation noise in electrons and as a percentage of the source, ADC noise, peak-pixel and saturation numbers, predicted magnitudes, the $\sigma(\text{EW})$ line criterion, the stack plan and the differential-photometry precision.

5.2 Scientific plots

The native Matplotlib window provides pan/zoom/save, manual axis limits (time axes accept ISO timestamps), autoscale per panel, and in spectroscopy a manual slider that walks the sampled times of the observing interval, updating the spectrum panels and the markers. The sky-path panel shows target, Sun (daytime only) and Moon (only above the horizon) in altitude–azimuth ($N=0^\circ$, $E=90^\circ$).

5.3 Practical notes

- The Moon’s effect is included automatically in `ing` sky mode whenever it is above the horizon; within $\sim 30^\circ$ of a bright Moon expect the sky several magnitudes brighter.
- In `fixed_ab` mode the entered sky brightness is used as-is at every time — use it when you have a measured value.

- Airmass and all results are deliberately unavailable below 5° altitude.
- Save an **instrument profile** once your telescope/detector column is right; it restores automatically at start-up and travels with its QE/response files.

6 Input file formats, in full

Everything the program reads is a plain text, JSON, XML or FITS file that you can inspect and produce yourself. This section specifies each format completely: the exact columns, their units, the parsing rules, and a literal example you can copy. Blank lines are ignored everywhere; lines beginning with # are comments where noted.

6.1 The template catalogue: `data/interpola.db.csv`

This comma-separated file is the master list of stellar templates; the template selector in the GUI is built from it. The first line is a header and is skipped. Every following line describes one template with exactly six fields:

#	Field	Meaning
1	nrow	Integer: the number of data rows in the spectrum file. Used only as a consistency check at load time (a mismatch larger than one row prints a warning, nothing more), so an approximate value is acceptable.
2	Vmag	Float: the <i>visual magnitude that the spectrum file represents</i> , called m_{v0} throughout the documentation. This is the heart of the calibration: the ETC multiplies the file by $10^{0.4m_{v0}}$ to bring it to the “visual zero” scale and then scales it to the magnitude you enter in the GUI. For an absolutely calibrated file (CALSPEC), m_{v0} is simply the synthetic V magnitude of the file itself: 0.026 for Vega, -26.75 for the Sun. For the BPGS library the files are already normalised to visual zero, hence $m_{v0} = 0$. If this number is wrong, every prediction for that template is wrong by the same factor — when in doubt, let <code>add_template.py</code> compute it (Sect. 7.2).
3	B-V	Float: the Johnson $B-V$ colour of the template. Informational: it is displayed in the selector and used by the template-validation button, but it does not enter the flux calibration.
4	name	The display name shown in the selector. Any text without a comma; leading/trailing spaces are stripped.
5	spt	The spectral type (A0V, G2V, M5III, ...). The search box of the selector matches against this field.
6	filename	Path of the spectrum file, <i>relative to the data directory</i> . Subdirectories are allowed and encouraged (bpgs/bpgs_92.ascii, imported/hd12345.fits).

A literal excerpt:

```
nrow ,Vmag ,B-V ,name ,spt ,filename
8839, 0.026, 0.000, Vega , A0V, bpgs/alpha_lyr_stis_004.ascii
```

```
1467, -26.750,    0.630, Sun      ,    G2V, bpgs/sun_reference_stis_001.ascii
481,   0.000,    0.310, HD 76483 ,    A3V, bpgs/bpgs_48.ascii
```

Fields are separated by commas; the spaces you see are padding for human readability and are stripped on parsing. A line with fewer than six fields, or with non-numeric values in the numeric fields, is silently skipped — a convenient way to comment a template out is therefore to put a letter in front of its `nrow`.

6.2 Template spectra

Two formats are accepted, distinguished by the file extension.

Two-column ASCII (any extension except `.fits/.fit`). Each line holds a wavelength in *Ångström* and a flux density F_λ in $\text{erg s}^{-1} \text{cm}^{-2} \text{Å}^{-1}$, separated by whitespace:

```
3500.0  4.215e-09
3502.5  4.198e-09
3505.0  4.187e-09
```

Parsing rules, all applied automatically at load: lines with fewer than two numeric tokens are skipped; rows with non-finite, zero or negative wavelength or flux are dropped; rows are sorted by wavelength; duplicate wavelengths are removed keeping the first occurrence. What matters physically is the *shape*: because the ETC rescales the spectrum to the magnitude you enter (via m_{v0} , previous subsection), a template in arbitrary flux units is perfectly usable as long as the relative calibration across wavelength is correct.

CALSPEC/STScI FITS tables (`.fits`, `.fit`). The standard format of the STScI calibration archives: a FITS binary table with a wavelength column named `WAVELENGTH` (or `WAVE`, `LAMBDA`, `WL`) and a flux column named `FLUX` (or `FLAM`, `SURF_BRIGHT`, `SB`), matched case-insensitively. The wavelength unit is read from the column’s `TUNIT` keyword and may be *Ångström* (the CALSPEC convention), nanometres or microns — conversion is automatic. Additional columns (`STATERROR`, `YSERROR`, `FWHM`, ...) are ignored. This one reader covers the CALSPEC standard-star archive, the solar-system surface-brightness atlas, the galactic emission-line atlas, the supernova/kilonova transient templates and the CLOUDY planetary-nebula grids without modification.

6.3 The filter list and profiles

`data/filters.list` names the available filters, one entry per line; `#` comments allowed. Each *logical* filter (say `Bessell.V`) requires two profile files, `Bessell.V_Vega` and `Bessell.V_AB`, holding the same transmission curve with the two photometric calibrations. Entries may be written with or without the `filters/` prefix and with or without the `.xml` extension — all four combinations resolve. Example:

```
filters/Bessell.V_Vega
filters/Bessell.V_AB
filters/SDSS.r_Vega
filters/SDSS.r_AB
```

Profile files (`data/filters/*.xml`) are VOTables in the SVO Filter Profile Service format. If you download a filter from SVO (<http://svo2.cab.inta-csic.es/theory/fps/>), take the “VOTable” link of the desired calibration and drop the file in unchanged. The parser requires four `PARAM` elements — `MagSys` (Vega or AB), `ZeroPoint` with `ZeroPointUnit` Jy, `DetectorType` (0 = energy counter, 1 = photon counter) and `WavelengthUnit` — plus the transmission table itself as

TR/TD rows of wavelength and transmission. All other metadata (pivot wavelength, FWHM, effective width, ...) is read when present and shown in the filter panel. For a filter that SVO does not carry — every amateur RGB or narrowband filter — build the pair with `make_filter_profile.py` from a plain transmission curve (Sect. 7.1); the generated files contain everything listed above.

BLANK.xml is the special “no filter” profile: unity transmission from 3000 to 26000 Å. Because an unfiltered measurement has no meaningful Vega calibration, it is defined on the AB scale for both selector positions. Spectroscopy selects it by default.

6.4 Detector and instrument curves

Four optional two-column text files describe your instrument beyond the scalar numbers in the GUI. All four share the same syntax: whitespace- or comma-separated columns, # comments, and a wavelength unit that you declare in the GUI (Ångström, nm or µm) rather than in the file — so a curve digitized from a datasheet in nanometres is used as-is by setting the unit selector to nm.

QE curve (data/qe.dat or per-profile). Wavelength and quantum efficiency as a *fraction* (0–1):

```
# ASI533MM wavelength[nm] QE
400 0.62
500 0.80
600 0.72
700 0.52
```

Values are clipped to [0, 1]; outside the tabulated range the QE is taken as zero, which is physically right for a sensor cut-off. For an OSC camera in single-channel mode, this file should be the *channel’s* effective QE: sensor QE multiplied by the colour-dye transmission of that channel, both published by the camera makers.

Optics response curve (optional). Same syntax; the wavelength-dependent throughput of everything between aperture and detector *excluding* the QE and the filter (mirror reflectivity, corrector and window transmission, ...). It multiplies the scalar efficiency from the GUI. If you have only the scalar, leave this file out.

Slit resolving-power curve (optional). Two columns: slit width in arcsec, measured resolving power R . When loaded, the measured curve overrides the nominal R for slit widths inside its range; a width outside the range is refused rather than extrapolated.

CMOS gain table (optional). Four columns: gain *setting* (the number you dial into the camera driver), conversion gain in e[−]/ADU, read noise in e[−] rms, full-well capacity in e[−]:

```
# ASI533MM: setting e/ADU RN FWC
0 3.64 3.55 50000
100 1.00 1.45 13700
200 0.32 1.20 4400
```

Selecting a row in the GUI fills the three detector fields together, so gain, noise and saturation always describe the same camera state. The numbers come from the maker’s published curves or from PhotonsToPhotos.

6.5 The atmospheric transmission FITS file

data/earth_atmospheric_transmission.fits, if present, replaces the built-in broad-band extinction table with a measured curve. Three layouts are accepted: a binary table with a wavelength column (WAVEAIR, WAVELENGTH, WAVE, LAMBDA, LAM or WL) and a direct transmission column (TRANSMISSION, TRANS, THROUGHPUT) holding fractions; the same with an EXTINCTION column in mag/airmass (the Paranal product), converted internally as $T = 10^{-0.4k\lambda}$; or a plain $2 \times N$ image whose wavelength unit is declared by CUNIT1. Whatever the source, the file must state *zenith* values: the ETC raises the curve to the current airmass itself, exactly once. Outside the file's wavelength range the edge values are extended (an atmosphere does not become opaque where a file ends).

6.6 Sites: etc_sites.json and observatories.json

Your personal site list is a JSON file next to the program:

```
{ "schema_version": 1,
  "sites": [
    { "name": "My backyard", "lat": 45.4064, "lon": 11.8768,
      "elev": 12, "utc_offset_h": 1, "timezone": "Europe/Rome",
      "sky_ab_mag_arcsec2": 19.8 } ] }
```

lat/lon are decimal degrees (north and east positive), elev metres, sky_ab_mag_arcsec2 the default value for the fixed-sky mode. You rarely edit this file by hand: **Save current site** writes the entry for you, and **Import site file** merges another user's file into yours. observatories.json holds the shipped defaults in a simpler flat format and is not meant to be edited.

6.7 Instrument profiles

Save instrument profile writes a JSON file containing every telescope/detector/spectroscopy setting of the middle columns, and copies the currently loaded QE, optics-response and slit- R curves beside it as <profile>_qe.dat, <profile>_throughput.dat and <profile>_slitres.dat, referenced by relative name — the set of files is portable as a folder. Loading a profile restores everything, including the curves; the most recently used profile is reloaded automatically at start-up. This is the mechanism to maintain one configuration per telescope+camera combination you own.

6.8 Horizon profile CSV

Written automatically by the horizon generator into data/horizons/horizon_lat<lat>_lon<lon>_r<km>km readable by any plotting tool and by Display horizon. A commented header records the site, then one row per azimuth:

```
# SPETC horizon profile
# latitude_deg: 45.848600
# longitude_deg: 11.569600
# radius_km: 10
# center_elevation_m: 1366.0
azimuth_deg, horizon_elevation_deg
0.0, 1.234
1.0, 1.198
```

Azimuth is degrees from North through East (N=0, E=90 — the same convention as every other azimuth in the program); the horizon elevation is the apparent altitude of the terrain in degrees, including the Earth-curvature drop and the standard $34'$ horizontal refraction, so a target is genuinely visible when its computed altitude exceeds the profile value at its azimuth.

6.9 Batch run descriptions

The headless mode (Sect. 7.3) is driven by a JSON file whose blocks mirror the GUI columns: telescope (mm and scalar efficiency), detector (the Detector constructor fields), atmosphere (airmass, seeing, PSF model, elevation), sky ("mode": "fixed_ab" with sky_mag, or "mode": "sqm" with sqm_v_mag_arcsec2), target (template name from the catalogue, magnitude, system, reference filter), and a photometry or spectroscopy block with the mode-specific settings. The two shipped examples in `examples/` are complete and commented by construction — copy one and edit the numbers.

7 Command-line tools

7.1 Calibrating your own filters: `make_filter_profile.py`

Amateur filters (Astrodon, Astronomik, Baader, Optolong RGB and narrowband) are not in the SVO database, but a transmission curve is all that is needed — the zero points are computed, not measured:

```
python3 make_filter_profile.py Astrodon.G astrodon_g.dat nm
```

reads a two-column wavelength/transmission file (0–1 or percent, auto-detected; unit Angstrom, nm or um), and writes `Astrodon.G_Vega.xml` and `Astrodon.G_AB.xml` into `data/filters/` in full SVO format: complete VO annotation, all characterising wavelengths, FWHM, effective width, the mean solar flux, and a Vega zero point computed synthetically from the bundled CALSPEC Vega spectrum — the same convention SVO uses (validated to $\lesssim 1\%$ against the shipped SVO profiles). Add the two printed lines to `data/filters.list` and the filter appears in the selector on the same photometric scale as the professional passbands.

7.2 Importing template spectra: `add_template.py`

Template spectra can be two-column ASCII or CALSPEC/STScI-style FITS tables (WAVELENGTH/FLUX; Angstrom, nm or micron units read from TUNIT). To add a single file or an entire directory — e.g. the whole CALSPEC archive folder:

```
python3 add_template.py current_calspec/
python3 add_template.py my_spectrum.fits "HD 12345" G2V
```

For each spectrum the tool copies the file under `data/imported/`, computes the synthetic Vega V magnitude (the catalogue `mv0`) and $B - V$ against the shipped Bessell profiles, cleans CALSPEC filename suffixes for the display name (`alpha_lyr_stis_011` $\rightarrow$ `alpha_lyr`), reports the V -band coverage, and appends the row to `data/interpolated.csv` — the file the GUI template list is built from. Press **Reload catalog** (or restart) and the new templates appear in the selector. Surface-brightness and arbitrary-flux files (planet atlas, supernova templates) are fully usable: the ETC rescales every template to your entered magnitude, so only the spectral shape and the computed `mv0` matter. Files that do not fully cover the V band are flagged; use V as the reference filter for those.

7.3 Headless runs: `spetc_batch.py`

```
python3 spetc_batch.py examples/batch_photometry.json out/
```

runs the same engines without the GUI from a JSON run description (two commented examples ship in `examples/`) and writes the full result CSV plus a self-contained **one-page HTML summary**: configuration, sky, key numbers (σ (EW) for spectroscopy, saturation, maximum unsaturated exposure), the stack plan, and an embedded S/N figure (S/N versus exposure for photometry; S/N versus wavelength

for spectroscopy). Batch sky modes are `fixed_ab` and `sqm`; the Moon and twilight terms need the GUI’s time machinery and are not applied, as the summary footer states. Useful for scripting parameter studies and for archiving a session plan.

8 Troubleshooting

Table 1: Common messages and their resolution.

Message / symptom	Cause and fix
“...covers X–Y Å, but the requested calculation needs...”	A curve (template, QE, filter, atmosphere) does not cover the requested wavelengths. Narrow the range or load data with wider coverage; the ETC never extrapolates silently.
“Target altitude is below 5 deg”	The selected time puts the target too low. Move the time or check the coordinates; the visibility plot shows when the target rises.
“The template has no positive flux in the selected magnitude bandpass”	Reference filter outside the template’s spectral range (e.g. <i>K</i> band with an optical-only spectrum). Choose a filter inside the template coverage.
Required exposure flagged as saturating	The target S/N needs a longer exposure than the brightest pixel allows. Use the reported maximum unsaturated exposure and stack frames.
S/N of a bright star lower than expected	Scintillation ceiling (physical, not an error). Stack short exposures; larger aperture and lower airmass raise the ceiling.
“Resolve name” fails	SIMBAD needs internet; type coordinates manually — nothing else requires the network.
Filters missing at start-up	<code>data/filters.list</code> not found or empty. Each logical filter needs its <code>_Vega</code> and <code>_AB</code> XML profiles; <code>BLANK.xml</code> in <code>data/filters/</code> enables the unfiltered response.
Slit width outside the calibrated range	A measured slit- <i>R</i> curve is loaded and the requested width lies outside it. Change the width or remove the curve.

If a run aborts with a Python traceback, the full text is shown in the error dialog: it names the offending input explicitly and is the first thing to attach to a bug report, together with `etc_user_config.json`.